

Web Frameworks

The Agentic AI Platform provides official wrapper components for popular frontend frameworks, simplifying the integration of the iframe into your applications. These components handle all the iframe communication, configuration, and lifecycle management automatically.

Supported Frameworks

Framework	Package	Status
React	<code>@trimble-agentic-external-npm-local/agentic-chat-react@0.0.2-rc.11</code>	 Available
Angular	<code>@trimble-agentic-external-npm-local/agentic-chat-angular@2.0.0</code>	 Available
Vue	<code>@trimble-agentic-external-npm-local/agentic-chat-vue@2.0.0</code>	 Available

For .NET desktop and mobile applications see .NET Wrappers.

When to Use Framework Components

Framework components are ideal when:

- 

 You're building with a supported framework (React, Angular, Vue)
- 

 You want a simpler, more declarative API
- 

 You prefer framework-native patterns (hooks, props, components)
- 

 You want automatic lifecycle management and cleanup

Consider using the base iframe SDK directly if:

-  You need maximum control over iframe communication
-  You're using an unsupported framework
-  You have complex custom messaging requirements
-  You want to have custom localization or configuration logic that doesn't fit the component props
-  You want to access the development environment features not yet exposed in stage or prod. This is not recommended, with the staging environment being the preferred initial environment.

React

The React wrapper provides components that integrate seamlessly with React applications, using hooks for state management and automatic cleanup on unmount.

Installation

```
npm install @trimble-agentic-external-npm-local/agentic-chat-react
```

Available Components

The package exports two main components:

Component	Description
<code>AgenticFullChat</code>	Full-featured Assist interface with agent selection and conversation history.
<code>AgenticMinimalChat</code>	Streamlined chat interface focused on the core conversation experience. No agent selection other than the default, and no conversation history other than the exposed threads in the agent's hero.

Basic Usage



```
import { AgenticFullChat } from '@trimble-agentic-external-npm-  
local/agentic-chat-react';  
import type { ChatUiEvent } from '@trimble-agentic-external-npm-  
local/agentic-chat-react';  
  
function App() {  
  // Your token provider function  
  const provideToken = async (): Promise<string> => {  
    // Return your Trimble Identity access token  
    return await getAccessToken();  
  };  
  
  // Optional: Handle chat events  
  const handleEvent = (event: ChatUiEvent) => {  
    console.log('Chat event:', event);  
  };  
  
  return (  
    <AgenticFullChat  
      provideToken={provideToken}  
      onEvent={handleEvent}  
      environment="prod"  
      theme="light"  
      agentId="your-agent-uuid"  
      chatInputConfiguration={{  
        buttons: [],  
        hideModelSelection: false,  
      }}  
    />  
  );  
}
```

Props Reference

Common Props (Both Components)

Prop	Type	Required	Description
<code>provideToken</code>	<code>() => Promise<string></code>		Async function that returns the TID access token

Prop	Type	Required	Description
<code>environment</code>	<code>'stage' 'prod'</code>	✓	Target environment
<code>theme</code>	<code>'light' 'dark'</code>	✓	UI theme
<code>agentId</code>	<code>string</code>	✓	UUID of the agent to use
<code>chatInputConfiguration</code>	<pre>{ buttons: ChatInputButton[], hideModelSelection: boolean }</pre>	✓	Configuration for custom buttons that will show up in the chat input dropdown.
<code>threadId</code>	<code>string</code>	✗	Optional thread UUID to load a specific conversation
<code>onEvent</code>	<pre>(event: ChatUiEvent) => void</pre>	✗	Event handler for chat events
<code>onBeforeRun</code>	<code>OnBeforeRunConfig</code>	✗	Configuration for pre-run hooks and local tools
<code>onBeforeRunTimeout</code>	<code>number</code>	✗	Optional timeout for onBeforeRun in milliseconds
<code>className</code>	<code>string</code>	✗	CSS class name applied to the iframe element

AgenticFullChat Additional Props

Prop	Type	Required	Description
<code>contentVariant</code>	<code>ContentVariants</code>	✗	Content variant (defaults to <code>ContentVariants.Chat</code>)

Complete Example

```
import { useState } from 'react';
import {
  AgenticFullChat,
  AgenticMinimalChat,
} from '@trimble-agentic-external-npm-local/agent-chat-react';
import type { ChatUiEvent } from '@trimble-agentic-external-npm-local/agent-chat-react';

type ChatVariant = 'minimal' | 'full';

function ChatPage() {
  const [variant, setVariant] = useState<ChatVariant>('full');
  const [theme, setTheme] = useState<'light' | 'dark'>('light');
  const [environment, setEnvironment] = useState<'stage' | 'prod'>('stage');

  const provideToken = async (): Promise<string> => {
    // Implement your token retrieval logic
    const response = await fetch('/api/auth/token');
    const { accessToken } = await response.json();
    return accessToken;
  };

  const handleEvent = (event: ChatUiEvent) => {
    console.log('Chat event:', event);
    // Handle events like message sent, agent response, errors, etc.
  };

  const chatInputConfiguration = {
    buttons: [],
    hideModelSelection: false,
  };

  const agentId = 'your-agent-uuid';
```

```

    return (
      <div style={{ height: '100vh', display: 'flex', flexDirection:
'column' }}>
        {/* Controls */}
        <div style={{ padding: '16px', borderBottom: '1px solid #ccc' }}>
          <select value={variant} onChange={e =>
setVariant(e.target.value as ChatVariant)}>
            <option value="full">Full</option>
            <option value="minimal">Minimal</option>
          </select>
          <select value={theme} onChange={e => setTheme(e.target.value as
'light' | 'dark')}>
            <option value="light">Light</option>
            <option value="dark">Dark</option>
          </select>
          <select value={environment} onChange={e =>
setEnvironment(e.target.value as 'stage' | 'prod')}>
            <option value="stage">Stage</option>
            <option value="prod">Prod</option>
          </select>
        </div>

        {/* Chat Component */}
        <div style={{ flex: 1, overflow: 'hidden' }}>
          {variant === 'minimal' ? (
            <AgenticMinimalChat
              provideToken={provideToken}
              onEvent={handleEvent}
              environment={environment}
              theme={theme}
              chatInputConfiguration={chatInputConfiguration}
              agentId={agentId}
            />
          ) : (
            <AgenticFullChat
              provideToken={provideToken}
              onEvent={handleEvent}
              environment={environment}
              theme={theme}
              chatInputConfiguration={chatInputConfiguration}
              agentId={agentId}
            />
          )}
        </div>
      </div>
    );
  }
}

```

```
export default ChatPage;
```

Using with Local Tools and Run Context (onBeforeRun)

The React components support the `onBeforeRun` configuration for registering local tools that can be invoked by the AI agent, as well as providing run context:

```
import { AgenticFullChat } from '@trimble-agentic-external-npm-local/agent-chat-react';
import type { OnBeforeRunConfig } from '@trimble-agentic-external-npm-local/agent-chat-react';

function AppWithLocalTools() {
  const provideToken = async () => {
    return await getAccessToken();
  };

  const onBeforeRun: OnBeforeRunConfig = {
    tools: {
      runTime: {
        get_current_weather: {
          definition: {
            name: 'get_current_weather',
            description: 'Get the current weather for a location',
            parameters: {
              type: 'object',
              properties: {
                location: { type: 'string', description: 'City name' },
              },
              required: ['location'],
            },
          },
          callback: async (args) => {
            const temp = 22;
            const conditions = "sunny"
            return JSON.stringify({ temp, conditions });
          },
          timeoutInMs: 10000,
        },
      },
      global: {},
    },
    runContext: {
      context: [
```

```

        { description: 'Current user', value: 'john.doe@example.com' },
        { description: 'Project ID', value: 'proj-12345' },
      ],
    },
  };

  return (
    <AgenticFullChat
      provideToken={provideToken}
      environment="prod"
      theme="light"
      agentId="your-agent-uuid"
      chatInputConfiguration={{ buttons: [], hideModelSelection: false
    }}

    onBeforeRun={onBeforeRun}
    onBeforeRunTimeout={60000} // 60 second timeout
  />
  );
}

```

Type Exports

The React package exports the components, as well as the type definitions from the base iframe SDK:

```

import {
  // Components
  AgenticFullChat,
  AgenticMinimalChat,
  // Types
  type AgenticChatProps,
  type AgenticFullChatProps,
  type AgenticMinimalChatProps,
  type OnBeforeRunConfig,
  type ChatUiEvent,
  type ChatInputButton,
} from "@trimble-agentic-external-npm-local/agentic-chat-react";

```

Angular

The Angular wrapper provides standalone components that integrate seamlessly with Angular applications, supporting Angular signals and automatic lifecycle management.

Installation

```
npm install @trimble-agentic-external-npm-local/agentic-chat-angular
```

Available Components

The package exports three main components:

Component	Description
<code>AgenticFullChat</code>	Full-featured Assist interface with agent selection and conversation history.
<code>AgenticMinimalChat</code>	Streamlined chat interface focused on the core conversation experience. No agent selection other than the default, and no conversation history other than the exposed threads in the agent's hero.
<code>AgenticNarrowChat</code>	Narrow chat variant suited to side panels and other constrained layouts.

Basic Usage

```
import { Component } from "@angular/core";
import { AgenticFullChat } from "@trimble-agentic-external-npm-local/agentic-chat-angular";
import type { ChatUiEvent } from "@trimble-agentic-external-npm-local/agentic-chat-angular";

@Component({
  selector: "app-chat",
  standalone: true,
  imports: [AgenticFullChat],
  template: `
    <agentic-full-chat
      [provideToken]="provideToken"
```

```

    [environment]='prod'
    [theme]='light'
    [agentId]="agentId"
    [chatInputConfiguration]="chatInputConfiguration"
    (onEvent)="handleEvent($event)"
  />
  `
,
})
export class ChatComponent {
  agentId = "your-agent-uuid";

  chatInputConfiguration = {
    hideModelSelection: false,
    buttons: [],
  };

  provideToken = async (): Promise<string> => {
    // Return your Trimble Identity access token
    return Promise.resolve("<TRIMBLE ACCESS TOKEN HERE>");
  };

  handleEvent(event: ChatUiEvent): void {
    console.log("Chat event:", event);
  }
}
```

Props Reference

Common Inputs (All Components)

Input	Type	Required	Description
provideToken	() => Promise<string>	✓	Async function that returns the TID access token. Either this or the deprecated getAccessToken must be supplied.

Input	Type	Required	Description
<code>environment</code>	<code>'stage' 'prod'</code>	✓	Target environment
<code>theme</code>	<code>'light' 'dark'</code>	✓	UI theme
<code>agentId</code>	<code>string</code>	✓	UUID of the agent to use
<code>chatInputConfiguration</code>	<pre>{ buttons: ChatInputButton[], hideModelSelection: boolean }</pre>	✓	Configuration for custom buttons that will show up in the chat input dropdown.
<code>getAccessToken</code>	<pre>() => Promise<string></pre>	✗	Deprecated — use <code>provideToken</code> instead. Still supported, but ignored when <code>provideToken</code> is also set.
<code>threadId</code>	<code>string</code>	✗	Optional thread UUID to load a specific conversation
<code>onBeforeRun</code>	<code>OnBeforeRunConfig</code>	✗	Configuration for pre-run hooks and local tools
<code>onBeforeRunTimeout</code>	<code>number</code>	✗	Timeout for <code>onBeforeRun</code> in milliseconds (default: 30000)

Input	Type	Required	Description
<code>className</code>	<code>string</code>	✗	CSS class name applied to the iframe element

AgenticFullChat Additional Props

Input	Type	Required	Description
<code>contentVariant</code>	<code>ContentVariants</code>	✗	Content variant (defaults to <code>ContentVariants.Chat</code>)

AgenticNarrowChat Additional Props

Input	Type	Required	Description
<code>productAgents</code>	<code>string[]</code>	✗	Optional list of product agent IDs forwarded to the embedded chat to populate its agent picker.

Common Outputs

Output	Type	Description
<code>onEvent</code>	<code>OutputRef<ChatUiEvent></code>	Event emitter for chat events

Complete example

```
import { Component } from "@angular/core";
import { CommonModule } from "@angular/common";
import { FormsModule } from "@angular/forms";
import {
  AgenticFullChat,
  AgenticMinimalChat,
  AgenticNarrowChat,
} from "@trimble-agentic-external-npm-local/agent-chat-angular";
import type { ChatUiEvent } from "@trimble-agentic-external-npm-
```

```
local/agent-ai-chat-angular";

type ChatVariant = "minimal" | "full" | "narrow";

@Component({
  selector: "app-chat-page",
  standalone: true,
  imports: [
    CommonModule,
    FormsModule,
    AgenticFullChat,
    AgenticMinimalChat,
    AgenticNarrowChat,
  ],
  template: `
    <div style="height: 100vh; display: flex; flex-direction: column;">
      <!-- Controls -->
      <div style="padding: 16px; border-bottom: 1px solid #ccc;">
        <select [(ngModel)]="variant" (change)="onVariantChange()">
          <option value="full">Full</option>
          <option value="minimal">Minimal</option>
          <option value="narrow">Narrow</option>
        </select>
        <select [(ngModel)]="theme">
          <option value="light">Light</option>
          <option value="dark">Dark</option>
        </select>
        <select [(ngModel)]="environment">
          <option value="stage">Stage</option>
          <option value="prod">Prod</option>
        </select>
      </div>

      <!-- Chat Component -->
      <div style="flex: 1; overflow: hidden;">
        @if (variant === "minimal") {
          <agent-ai-minimal-chat
            [provideToken]="provideToken"
            [environment]="environment"
            [theme]="theme"
            [agentId]="agentId"
            [chatInputConfiguration]="chatInputConfiguration"
            (onEvent)="handleEvent($event)"
          />
        } @else if (variant === "narrow") {
          <agent-ai-narrow-chat
            [provideToken]="provideToken"

```

```

        [environment]="environment"
        [theme]="theme"
        [agentId]="agentId"
        [chatInputConfiguration]="chatInputConfiguration"
        (onEvent)="handleEvent($event)"
      />
    } @else {
      <agentic-full-chat
        [provideToken]="provideToken"
        [environment]="environment"
        [theme]="theme"
        [agentId]="agentId"
        [chatInputConfiguration]="chatInputConfiguration"
        (onEvent)="handleEvent($event)"
      />
    }
  </div>
</div>
` ,
))
export class ChatPageComponent {
  variant: ChatVariant = "full";
  theme: "light" | "dark" = "light";
  environment: "stage" | "prod" = "stage";
  agentId = "your-agent-uuid";

  chatInputConfiguration = {
    buttons: [],
    hideModelSelection: false,
  };

  provideToken = async (): Promise<string> => {
    // Return your Trimble Identity access token
    return Promise.resolve("<TRIMBLE ACCESS TOKEN HERE>");
  };

  handleEvent(event: ChatUiEvent): void {
    console.log("Chat event:", event);
    // Handle events like message sent, agent response, errors, etc.
  }

  onVariantChange(): void {
    console.log("Variant changed to:", this.variant);
  }
}

```

Using with Local Tools and Run Context (onBeforeRun)

The Angular components support the `onBeforeRun` configuration for registering local tools that can be invoked by the AI agent, as well as providing run context:

```
import { Component } from "@angular/core";
import { AgenticFullChat } from "@trimble-agentic-external-npm-local/agentic-chat-angular";
import type {
  ChatUiEvent,
  OnBeforeRunConfig,
} from "@trimble-agentic-external-npm-local/agentic-chat-angular";

@Component({
  selector: "app-local-tools-chat",
  standalone: true,
  imports: [AgenticFullChat],
  template: `
    <agentic-full-chat
      [provideToken]="provideToken"
      [environment]="environmentName"
      [theme]="theme"
      [agentId]="agentId"
      [chatInputConfiguration]="chatInputConfiguration"
      [onBeforeRun]="onBeforeRun"
      [onBeforeRunTimeout]="60000"
      (onEvent)="handleEvent($event)"
    />
  `,
})
export class LocalToolsChatComponent {
  environmentName: "stage" | "prod" = "prod";
  theme: "light" | "dark" = "light";
  agentId = "your-agent-uuid";

  chatInputConfiguration = {
    buttons: [],
    hideModelSelection: false,
  };

  provideToken = async (): Promise<string> => {
    // Implement your token retrieval logic
    return Promise.resolve("<TRIMBLE ACCESS TOKEN HERE>");
  };

  onBeforeRun: OnBeforeRunConfig = {
```

```
tools: {
  runTime: {
    get_current_weather: {
      definition: {
        name: "get_current_weather",
        description: "Get the current weather for a location",
        parameters: {
          type: "object",
          properties: {
            location: { type: "string", description: "City name" },
          },
          required: ["location"],
        },
      },
      callback: async (args: any) => {
        return JSON.stringify({
          temperature: 22,
          conditions: "sunny",
        });
      },
      timeoutInMs: 10000,
    },
  },
  global: {},
},
runContext: {
  context: [
    { description: "Current user", value: "john.doe@example.com" },
    { description: "Project ID", value: "proj-12345" },
  ],
},
};

handleEvent(event: ChatUiEvent): void {
  console.log("Chat event:", event);
}

// Helper function (replace with your real API call)
async function fetchWeather(
  location: string,
): Promise<{ temp: number; conditions: string }> {
  // Example: Replace with your actual API logic
  return { temp: 22, conditions: "Sunny" };
}
```


Type Exports

The Angular package exports the components, as well as the type definitions from the base iframe SDK:

```
import {
  AgenticFullChat,
  AgenticMinimalChat,
  AgenticNarrowChat,
} from "@trimble-agentic-external-npm-local/agentic-chat-angular";
import type {
  ChatUiEvent,
  OnBeforeRunConfig,
  ChatInputButton,
} from "@trimble-agentic-external-npm-local/agentic-chat-angular";
```

Vue

The Vue wrapper provides components that integrate seamlessly with Vue applications, using the Composition API for state management and automatic cleanup on unmount.

Installation

```
npm install @trimble-agentic-external-npm-local/agentic-chat-vue
```

Available Components

The package exports three main components:

Component	Description
AgenticFullChat	Full-featured Assist interface with agent selection and conversation history.
AgenticMinimalChat	Streamlined chat interface focused on the core conversation experience. No agent selection other than the default, and no

Component	Description
	conversation history other than the exposed threads in the agent's hero.
AgenticNarrowChat	Narrow chat variant suited to side panels and other constrained layouts.

Basic Usage

```
<script setup lang="ts">
import { AgenticFullChat } from "@trimble-agentic-external-npm-local/agent-ic-chat-vue";
import type { ChatUiEvent } from "@trimble-agentic-external-npm-local/agent-ic-chat-vue";

const provideToken = async (): Promise<string> => {
  // Implement your token retrieval logic here (e.g., call your auth API)
  return "YOUR_ACCESS_TOKEN";
};

const handleEvent = (event: ChatUiEvent): void => {
  console.log("Chat event:", event);
};

const agentId = "your-agent-uuid";
</script>

<template>
  <AgenticFullChat
    :provideToken="provideToken"
    environment="prod"
    theme="light"
    :agentId="agentId"
    :chatInputConfiguration="{ buttons: [], hideModelSelection: false }"
    :onEvent="handleEvent"
  />
</template>
```

Props Reference

Common Props (All Components)

Prop	Type	Required	Description
<code>provideToken</code>	<code>() => Promise<string></code>	✓	Async function that returns the TID access token. Either this or the deprecated <code>getAccessToken</code> must be supplied.
<code>environment</code>	<code>'stage' 'prod'</code>	✓	Target environment
<code>theme</code>	<code>'light' 'dark'</code>	✓	UI theme
<code>agentId</code>	<code>string</code>	✓	UUID of the agent to use
<code>chatInputConfiguration</code>	<code>{ buttons: ChatInputButton[], hideModelSelection: boolean }</code>	✓	Configuration for custom buttons that will show up in the chat input dropdown.
<code>getAccessToken</code>	<code>() => Promise<string></code>	✗	Deprecated — use <code>provideToken</code> instead. Still supported, but ignored when <code>provideToken</code> is also set.
<code>threadId</code>	<code>string</code>	✗	Optional thread UUID to load a

Prop	Type	Required	Description
			specific conversation
<code>onEvent</code>	<code>(event: ChatUiEvent) => void</code>	✗	Event handler for chat events
<code>onBeforeRun</code>	<code>OnBeforeRunConfig</code>	✗	Configuration for pre-run hooks and local tools
<code>onBeforeRunTimeout</code>	<code>number</code>	✗	Optional timeout for onBeforeRun in milliseconds
<code>className</code>	<code>string</code>	✗	CSS class name applied to the iframe element

AgenticFullChat Additional Props

Prop	Type	Required	Description
<code>contentVariant</code>	<code>ContentVariants</code>	✗	Content variant (defaults to <code>ContentVariants.Chat</code>)

AgenticNarrowChat Additional Props

Prop	Type	Required	Description
<code>productAgents</code>	<code>string[]</code>	✗	Optional list of product agent IDs forwarded to the embedded chat to populate its agent picker.

Complete Example

```

<script setup lang="ts">
import { ref } from "vue";
import {
  AgenticFullChat,
  AgenticMinimalChat,
  AgenticNarrowChat,
} from "@trimble-agentic-external-npm-local/agent-ic-chat-vue";
import type { ChatUiEvent } from "@trimble-agentic-external-npm-
local/agent-ic-chat-vue";

type ChatVariant = "minimal" | "full" | "narrow";

const variant = ref<ChatVariant>("minimal");
const theme = ref<"light" | "dark">("light");
const environment = ref<"stage" | "prod">("stage");
const agentId = "your-agent-uuid";

const provideToken = async (): Promise<string> => {
  // Implement your token retrieval logic here (e.g., call your auth
API)
  return "YOUR_ACCESS_TOKEN";
};

const handleEvent = (event: ChatUiEvent): void => {
  console.log("Chat event:", event);
  // Handle events like message sent, agent response, errors, etc.
};

const chatInputConfiguration = {
  buttons: [
    { id: "button-1", label: "Button 1" },
    { id: "button-2", label: "Button 2" },
  ],
  hideModelSelection: false,
};
</script>

<template>
  <div id="app" :class="theme === 'dark' ? 'theme-dark' : 'theme-
light'">
    <div id="controls">
      <label>
        Variant:
        <select class="variant" v-model="variant">
          <option value="minimal">Minimal</option>
          <option value="full">Full</option>

```

```
      <option value="narrow">Narrow</option>
    </select>
  </label>

  <label>
    Theme:
    <select class="variant" v-model="theme">
      <option value="light">Light</option>
      <option value="dark">Dark</option>
    </select>
  </label>

  <label>
    Environment:
    <select class="variant" v-model="environment">
      <option value="stage">Stage</option>
      <option value="prod">Prod</option>
    </select>
  </label>
</div>

<div id="chat" style="height: 100dvh; width: 100dvh">
  <AgenticMinimalChat
    v-if="variant === 'minimal'"
    :provideToken="provideToken"
    :environment="environment"
    :theme="theme"
    :agentId="agentId"
    :chatInputConfiguration="chatInputConfiguration"
    :onEvent="handleEvent"
  />

  <AgenticNarrowChat
    v-else-if="variant === 'narrow'"
    :provideToken="provideToken"
    :environment="environment"
    :theme="theme"
    :agentId="agentId"
    :chatInputConfiguration="chatInputConfiguration"
    :onEvent="handleEvent"
  />

  <AgenticFullChat
    v-else
    :provideToken="provideToken"
    :environment="environment"
    :theme="theme"
```

```
        :agentId="agentId"
        :chatInputConfiguration="chatInputConfiguration"
        :onEvent="handleEvent"
      />
    </div>
  </div>
</template>

<style scoped>
#app {
  height: 100vh;
  width: 100vw;
  display: flex;
  flex-direction: column;
}

#app.theme-dark {
  background-color: var(--modus-wc-color-gray-10, #1a1a1a);
  color: var(--modus-wc-color-white, #fff);
}

#app.theme-light {
  background-color: var(--modus-wc-color-base-page, #f5f5f5);
  color: var(--modus-wc-color-black, #000);
}

#controls {
  padding: 16px;
  border-bottom: 1px solid var(--modus-wc-color-base-300, #ccc);
  display: flex;
  gap: 16px;
  align-items: center;
  flex-wrap: wrap;
}

.variant {
  margin-left: 8px;
  border: 1px solid var(--modus-wc-color-base-300, #ccc);
  background-color: var(--modus-wc-color-base-100, #fff);
  color: inherit;
}

#chat {
  flex: 1;
  overflow: hidden;
```

```
}  
</style>
```

Using with Local Tools and Run Context (onBeforeRun)

The Vue components support the `onBeforeRun` configuration for registering local tools that can be invoked by the AI agent, as well as providing run context:

```
<script setup lang="ts">  
import { AgenticFullChat } from "@trimble-agentic-external-npm-  
local/agentic-chat-vue";  
import type { OnBeforeRunConfig } from "@trimble-agentic-external-npm-  
local/agentic-chat-vue";  
  
const provideToken = async (): Promise<string> => {  
  // Implement your token retrieval logic here (e.g., call your auth  
  API)  
  return "YOUR_ACCESS_TOKEN";  
};  
  
const onBeforeRun: OnBeforeRunConfig = {  
  tools: {  
    runTime: {  
      get_current_weather: {  
        definition: {  
          name: "get_current_weather",  
          description: "Get the current weather for a location",  
          parameters: {  
            type: "object",  
            properties: {  
              location: { type: "string", description: "City name" },  
            },  
            required: ["location"],  
          },  
        },  
      },  
      callback: async (args) => {  
        return JSON.stringify({  
          temperature: 22,  
          conditions: "sunny",  
        });  
      },  
      timeOutInMs: 10000,  
    },  
  },  
  global: {},  
};
```



```
    },
    runContext: {
      context: [
        { description: "Current user", value: "john.doe@example.com" },
        { description: "Project ID", value: "proj-12345" },
      ],
    },
  };
</script>

<template>
  <div id="chat" style="height: 100dvh; width: 100dvh">
    <AgenticFullChat
      :provideToken="provideToken"
      environment="prod"
      theme="light"
      agentId="your-agent-uuid"
      :chatInputConfiguration="{ buttons: [], hideModelSelection: false
    }"

      :onBeforeRun="onBeforeRun"
      :onBeforeRunTimeout="60000"
    />
  </div>
</template>
```

Type Exports

The Vue package exports the components, as well as the type definitions from the base iframe SDK:

```
import {
  AgenticFullChat,
  AgenticMinimalChat,
  AgenticNarrowChat,
} from "@trimble-agentic-external-npm-local/agentic-chat-vue";
import type {
  ChatUiEvent,
  OnBeforeRunConfig,
  ChatInputButton,
} from "@trimble-agentic-external-npm-local/agentic-chat-vue";
```

Migration from Base iframe SDK

If you're currently using the base iframe SDK directly and want to migrate to the framework specific components, the process is straightforward. The framework components are designed to abstract away the iframe communication and lifecycle management, so you can replace your existing implementation with the new components while keeping your configuration and token logic mostly intact:

Before (Base iframe SDK)

```
import { useEffect, useRef } from 'react';
import {
  listenToChatUi,
  CHAT_UI_URLS,
  type ChatUiConfiguration,
  ContentVariants,
  ChatUiVariants,
} from '@trimble-agentic-external-npm-local/agentic-platform-sdk-iframe-typescript';

function Chat() {
  const iframeRef = useRef<HTMLIFrameElement>(null);
  const environment = 'prod';
  const iframeUrl = CHAT_UI_URLS[environment];

  useEffect(() => {
    if (!iframeRef.current) return;

    const getConfig = (): ChatUiConfiguration => ({
      environment,
      agentId: 'your-agent-uuid',
      uiConfig: {
        theme: 'light',
        contentVariant: ContentVariants.Chat,
        variant: ChatUiVariants.Full,
        chatInput: { buttons: [], hideModelSelection: false },
      },
      localization: {},
    });

    const getToken = async () => token;

    const unsubscribe = listenToChatUi(
      iframeRef.current,
```

```
        iframeUrl,  
        getConfig,  
        getToken  
    );  
  
    return () => unsubscribe();  
}, []);  
  
    return <iframe ref={iframeRef} src={iframeUrl} style={{ width:  
'100%', height: '100%' }} />;  
}
```

After (React Component)

```
import { AgenticFullChat } from '@trimble-agentic-external-npm-  
local/agentic-chat-react';  
  
function Chat() {  
    const provideToken = async () => token;  
  
    return (  
        <AgenticFullChat  
            provideToken={provideToken}  
            environment="prod"  
            theme="light"  
            agentId="your-agent-uuid"  
            chatInputConfiguration={{ buttons: [], hideModelSelection: false  
        }}  
        />  
    );  
}
```